

その日時は Instant ? LocalDateTime ?

2026/05/29 DroidKaigi.collect { #30@Tokyo }



自己紹介

- ❑ どぎー
- ❑ 元？ゆめみ
- ❑ Android App Developer
- ❑ フジロックが楽しみ！7月末！

X @Kaito_Dogi

GitHub @Kaito-Dogi



**Instant と LocalDateTime
どう使い分けていますか？**

本 LT のゴール：

**Instant と LocalDateTime の
使い分けを整理する**

kotlinx.datetime とは

- ❑ KMP 対応の日時を扱うライブラリ
- ❑ ISO 8601（国際規格）に準拠
- ❑ 日時を表現する型や、
日時の変換操作や計算操作を提供



<https://github.com/Kotlin/kotlinx-datetime>

Instant とは

- ❑ 時間軸上の1点
- ❑ v0.7.0 以降では kotlin.time に移行（Kotlin v2.3 以降で Stable）*

```
// 端末の現在時刻から Instant を取得
val instantFromSystemClock = Clock.System.now()

// ISO 8601 準拠の文字列を Instant にパース
val instantFromUtcOffset = Instant.parse("2026-05-29T19:00:00+09:00")
```

* <https://kotlinlang.org/docs/whatsnew23.html#standard-library>

Instant とは

Use `kotlin.time.Instant` to represent a timestamp of the event that had already happened in the past (like a timestamp of a log entry) or will definitely happen in a well-defined instant of time in the future not far away from now (like an order confirmation deadline in 1 hour from now) .

<https://github.com/Kotlin/kotlinx-datetime#type-use-cases>

Instant とは

Use `kotlin.time.Instant` to represent a point in time (like a timestamp of a log entry) or will definitely happen in a well-defined instant of time in the future not far away from now (like an order confirmation deadline in 1 hour from now).

ログのタイムスタンプ

1時間後の締め切り

<https://github.com/Kotlin/kotlinx-datetime#type-use-cases>

Instant の罠

- ❑ 未来の日時を扱うにはリスクが伴う

<https://codeblog.jonskeet.uk/2019/03/27/storing-utc-is-not-a-silver-bullet/>

Instant の罠

- ❑ 未来の日時を扱うにはリスクが伴う
- ❑ **タイムゾーンが変わる可能性がある**（サマータイムの導入など）
- ❑ あらかじめ Instant で表現していると、日時への変換時にズれる
 - ❑ 例：元々の日本時刻の「2026/05/29 19:00」が「20:00」に

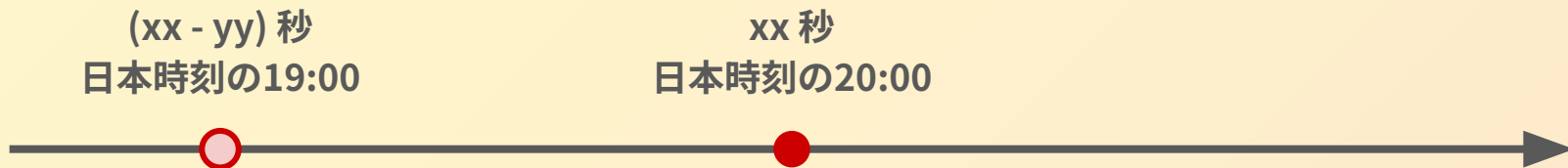
xx 秒
日本時刻の19:00



<https://codeblog.jonskeet.uk/2019/03/27/storing-utc-is-not-a-silver-bullet/>

Instant の罠

- ❑ 未来の日時を扱うにはリスクが伴う
- ❑ **タイムゾーンが変わる可能性がある**（サマータイムの導入など）
- ❑ あらかじめ Instant で表現していると、日時への変換時にズれる
 - ❑ 例：元々の日本時刻の「2026/05/29 19:00」が「20:00」に



<https://codeblog.jonskeet.uk/2019/03/27/storing-utc-is-not-a-silver-bullet/>

**Instant で未来の日時を扱うには
リスクが伴う**

LocalDateTime とは

- ❑ 壁掛け時計
- ❑ どこかの日付と時刻を表現する
- ❑ タイムゾーンをもたない

```
val localDateTime = LocalDateTime(2026, 5, 29, 19, 0)
```

LocalDateTime の罠

- ❏ 「2026/05/29 19:00」 はいつ？

LocalDateTime の罠

- ❑ 「2026/05/29 19:00」はいつ？
- ❑ タイムゾーンがないと、どこの日時が分からない
- ❑ LocalDateTime で時間軸上の1点を定めるには、タイムゾーンが必要

Call opens at 10:00 AM

16 Apr 2026

Call closes at 11:59 PM

17 May 2026

Call closes in **Tokyo Standard Time (UTC+09:00)** timezone.

<https://sessionize.com/droidkaigi2026/>

LocalDateTime とは

Use `LocalDateTime` to represent a time of the event that is scheduled to happen in the far future at a certain local time (like a scheduled meeting in a few months from now). You'll have to keep track of the `TimeZone` of the scheduled event separately. Try to avoid converting future events to `Instant` in advance, because time zone rules might change unexpectedly in the future.

<https://github.com/Kotlin/kotlinx-datetime#type-use-cases>

LocalDateTime とは

Use LocalDateTime to represent a time of the event that is scheduled to happen in the far future at a certain local time (like a scheduled meeting in a few months from now). You'll have to keep track of the TimeZone of the scheduled event separately. Try to avoid converting future events to Instant in advance because time zone rules might change unexpectedly in the future.

数ヶ月後の予定

<https://github.com/Kotlin/kotlinx-datetime#type-use-cases>

LocalDateTime はタイムゾーンをもたない

Instant と LocalDateTime の使い分け

	表現する情報	ユースケース
Instant	時間軸上の1点	過去の瞬間や近い将来 ログや登録日時、 数分後、数時間後の 締め切り
LocalDateTime	どこかの日時	未来の予定 イベントの開始時刻

Instant ↔ LocalDateTime の変換

- ❑ いずれもタイムゾーンが必要

- ❑ Instant → LocalDateTime : どの日時かを決めるため

- ❑ LocalDateTime → Instant : どの日時かを知るため

```
val timeZone = TimeZone.of("Asia/Tokyo")

val instant = Clock.System.now()
val localDateTimeFromInstant = instant.toLocalDateTime(timeZone)

val localDateTime = LocalDateTime(2026, 5, 29, 19, 0)
val instantFromLocalDateTime = localDateTime.toInstant(timeZone)
```

kotlinx.datetime の設計思想

However, there are convenience operations that take, for example, a physical instant and perform a calendar-based adjustment (such as adding a month); all such operations explicitly take time-zone information as a parameter to clearly state that their result depends on the civil time zone rules which are subject to change at any time.

<https://github.com/Kotlin/kotlinx-datetime#design-overview>

kotlinx.datetime の設計思想

However, there are convenience operations that take, for example, a physical instant and perform a calendar-based adjustment (such as adding a month); all such operations explicitly take time-zone information as a parameter to clearly state that their result depends on the civil time to change at any time.

すべての操作でタイムゾーンが必要

<https://github.com/Kotlin/kotlinx-datetime#design-overview>

まとめ

- ❑ Instant
 - ❑ 時間軸上の1点
 - ❑ 過去の瞬間や近い将来を表すのに適している
- ❑ LocalDateTime
 - ❑ どこかの日時（タイムゾーンをもたない）
 - ❑ 未来の予定を表すのに適している
- ❑ Instant ↔ LocalDateTime の変換ではタイムゾーンが必要

参考資料

- ❑ <https://github.com/Kotlin/kotlinx-datetime>
- ❑ <https://kotlinlang.org/api/core/kotlin-stdlib/kotlin.time/-instant/>
- ❑ <https://kotlinlang.org/docs/whatsnew23.html#standard-library>
- ❑ <https://codeblog.jonskeet.uk/2019/03/27/storing-utc-is-not-a-silver-bullet/>